

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Operating System (BCSE303P)

Course Project – End-to-End Project Guide

L15+L16

Faculty: Dr. Anisha M. Lal

Adaptive CPU Scheduling Using Predicted Burst Lengths

Prepared by

Shivaansh Aggarwal	24BCE0883
Devansh Sharma	24BCE0717

Contents

1. The project in one page
2. Concepts used in the project
3. Architecture
4. What happens during one simulation run
5. Code walkthrough, module by module
6. The prediction model
7. The adaptive scheduler in detail, with worked examples
8. Experiments and evaluation pipeline
9. Robustness to prediction errors: the overrun guard, W7 and statistics
10. Results and what they mean
11. How to run everything
12. Where everything is saved
13. Limitations and future work
14. Questions to be ready for

1. The project in one page

What it is. A CPU scheduler that predicts how long each process will run on its next CPU burst, and uses that prediction twice: to decide **which** process runs next, and to decide **how long** it may run (the time quantum). The time quantum is recalculated continuously while the system runs, instead of being fixed by hand.

Why. SJF gives the lowest average waiting time but needs the next burst length, which nobody knows in advance. The usual estimate, exponential averaging, reacts slowly when a process changes behaviour. Round Robin needs a quantum, and the textbook rule for choosing one (about 80% of bursts should finish within one quantum) is normally applied once by a person and never revisited.

How it was built. Everything is in Python. A discrete-event simulator models one CPU with processes that alternate CPU bursts and I/O. Eleven scheduler configurations run on exactly the same workloads: FCFS, SJF, SRTF, Round Robin (four quanta), MLFQ, an oracle that knows the future, and two versions of the proposed scheduler. A gradient-boosting model, trained on generated process histories, predicts the next burst.

Result in one line. On the mixed workload, the realistic case, the proposed scheduler cuts average waiting time by **27%** compared with SJF using exponential averaging. It is better on all 10 seeds, closes **54%** of the gap to the oracle, lowers the worst-case wait by **23%**, and uses

61% fewer context switches than Round Robin with a 5 ms quantum. It does not help on workloads made only of short random bursts, and the documents say so openly.

What happens when the prediction is wrong. A burst that runs longer than predicted keeps its short estimate, so with time slicing it returns to the ready queue looking short again and again. An **overrun guard** fixes this: once a burst clearly outruns its prediction, its estimated time left becomes max(prediction, 0.25 × CPU time used). On the intentionally adversarial **W7 bait-and-switch** workload the guard cuts the average wait by **21.5%** and interactive waiting by **92%**, with no significant change on W1–W5. MLFQ still has a lower average wait on W7, and a second idea, persistent ageing, made average waits worse and was not kept. Every main claim is checked with paired statistical tests.

2. Concepts used in the project

Term	Meaning in this project
CPU burst / I/O burst	A period of computing / a period of waiting for I/O. Every process here is *cpu, io, cpu, io, ..., cpu*.
Waiting time (W)	Time spent in the ready queue: turnaround − CPU time − I/O time. The main metric.
Turnaround time (T)	Completion time − arrival time.
Response time (R)	First time on the CPU − arrival time. Matters for interactive programs.
Context switch	Moving the CPU to a different process. Costs 0.1 ms in the simulator.
FCFS	Runs in arrival order. Convoy effect: 24, 3, 3 ms bursts give 17 ms average wait.
SJF / SRTF	Shortest burst first; SRTF also preempts when a shorter job arrives. Optimal on average, but needs burst lengths.
Exponential averaging (EMA)	$\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$, $\alpha = 0.5$. The classical burst estimate.
Round Robin (RR), quantum	Each process gets at most q ms, then goes to the back of the queue.
MLFQ	Multilevel feedback queue: processes that use their whole quantum drop a level; periodic boost avoids starvation.
Starvation, ageing	A process waiting forever; ageing improves a process's priority the longer it waits.
Oracle	SRTF that knows every true burst length: a reference that uses information no real scheduler has. Not a lower bound when processes have many bursts (FCFS beats it on W5).
MAE	Mean absolute error of the burst prediction, in ms.
Seed	A number that fixes the random workload, so a run can be repeated exactly. Seeds 1000–1004 train the model, 100–102 tune parameters, 0–9 evaluate.

Overrun guard	Once a burst has run longer than its prediction × the median observed actual/predicted ratio, its time left is taken as max(prediction, 0.25 × CPU time used), so it stops looking like a short job.
Noisy oracle	A diagnostic predictor that reads the true burst and multiplies it by exp(σZ). Used only to measure how much prediction quality matters; never a real scheduler.
W7 bait-and-switch	An intentionally adversarial workload: 30% of processes run many ~1 ms bursts and then one ~400 ms burst. They are 76% of the CPU demand.
Paired test, CI, Holm	Each comparison uses the same 10 seeds for both schedulers. 95% confidence intervals come from a bootstrap; p-values from the exact Wilcoxon signed-rank test, Holm-corrected. Significant means Holm $p < 0.05$.

3. Architecture

The key design decision is that the **simulation engine never makes a scheduling decision**. It owns the clock, the event queue and the process control blocks (PCBs), and whenever a decision is needed it calls the **policy interface**. Any scheduler that implements the interface can be plugged in. This guarantees that every scheduler is tested on identical arrivals, bursts, I/O times and switch costs, so differences in the results come only from the decisions.

Inside the adaptive scheduler, the **feature extractor** turns a process's history into 12 numbers, the **burst predictor** estimates the next burst, the **classifier** labels the process type, the **ready-queue ordering** applies the ageing formula (and, in Adaptive-ML-G, the overrun guard), and the **quantum controller** recomputes the time slice. When a burst completes, the **feedback loop** records the error and stores the example for refitting. After a run, the **metrics collector** computes all measures and the **analysis** step produces tables and figures.

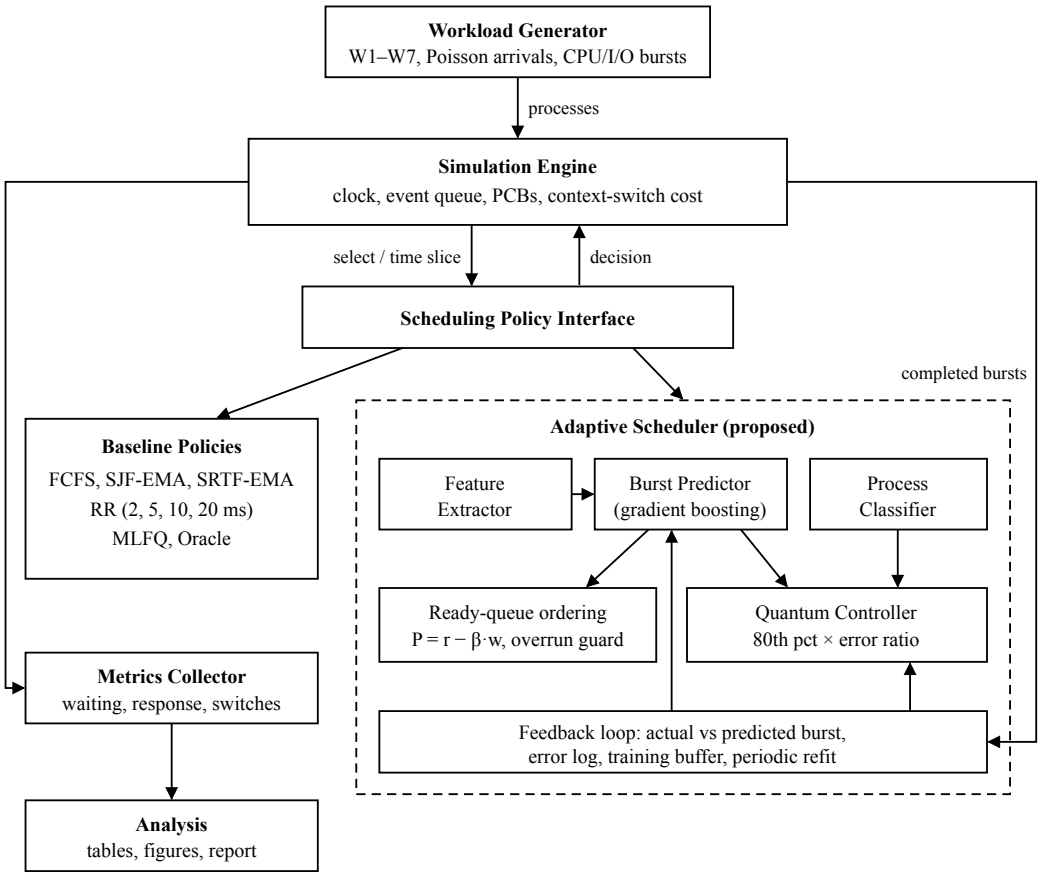


Fig. 1. System architecture

4. What happens during one simulation run

Following one process through the engine (`scheduler/engine.py`) makes the whole system easy to understand:

1. **Start.** The engine puts an `ARRIVAL` event for every process into a priority queue ordered by time.
2. **Arrival.** When an arrival event is popped, the process's first burst is loaded (`_start_burst`) and it joins the ready queue (`_make_ready`). The policy's `on_ready` hook runs; for the adaptive scheduler, this is where the next burst is predicted.
3. **Preemption check.** For preemptive policies (SRTF, MLFQ, oracle), the engine asks `should_preempt` whether the newcomer should interrupt the running process.
4. **Dispatch.** After all events at the current instant are handled, if the CPU is idle the engine calls `select` to pick a process and `time_slice` to get its quantum. If the process differs from the last one, a 0.1 ms switch cost is charged. A `SLICE_END` event is scheduled at *start + min(quantum, remaining burst)*.
5. **Slice end.** If the burst is finished, `on_burst_complete` runs (feedback loop), and an `IO_DONE` event is scheduled after the I/O time. If the quantum ran out instead,

`on_quantum_expired` runs and the process goes back to the ready queue.

6. **I/O done.** The next burst is loaded and the process becomes ready again. Steps 2–6 repeat.
7. **Finish.** After the last burst, the completion time is recorded. When the event queue is empty, the engine returns a `SimResult` with all PCBs, the timeline and the switch count.

Two details keep the engine correct. First, all events at the same time are processed before a decision is made, so processes arriving together are all considered (this is what makes the SJF example give exactly 3 ms). Second, when a slice is cut short by preemption, its pending `SLICE_END` event is cancelled with a token, so it cannot fire later by mistake.

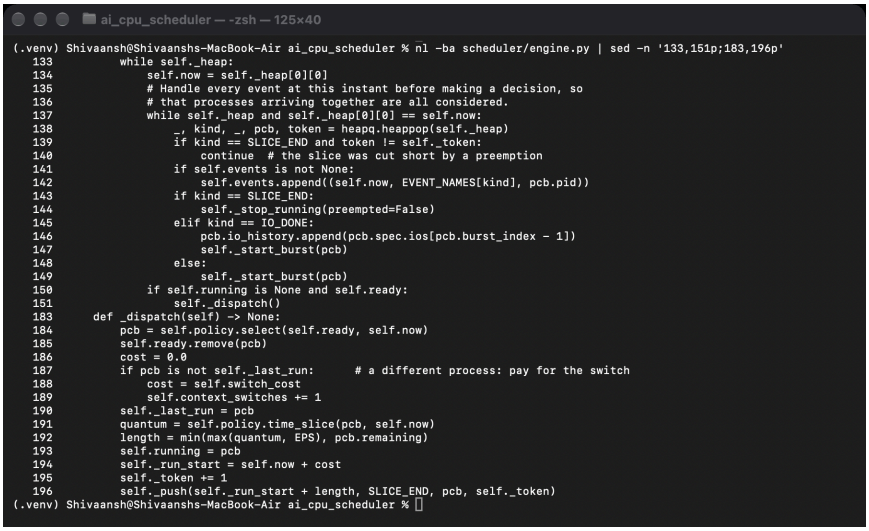


Fig. 2. The event loop and dispatch in scheduler/engine.py (Terminal, with line numbers)

5. Code walkthrough, module by module

File	What is inside
scheduler/workload.py	`ProcessSpec` (one process: arrival, bursts, I/O waits, true class of each burst). `make_process` draws one process of a type. `generate_workload(profile, seed)` builds W1–W6 with Poisson arrivals at 85% load. `shifting_workload` builds the changing workload used for the quantum figure. `simple_processes` builds textbook examples. The adversarial W7 and its `bait` process type are kept in `ADVERSARIAL_PROFILES`, separate from W1–W6.
scheduler/engine.py	`PCB` (run-time state), `Policy` (the interface: `on_ready, select, time_slice, should_preempt, on_quantum_expired, on_preempt, on_burst_complete, stats`), `Simulator` (event loop), `SimResult`.
scheduler/baselines.py	`FCFS`, `RoundRobin`, `SJF`, `SRTF`, `MLFQ`, `Oracle`; `PredictivePolicy` (shared code for policies that use a predictor); `estimated_remaining`.
scheduler/features.py	`ema_estimate`, `extract_features` (the 12 features), `build_dataset` (one row per burst for training).

scheduler/predictor.py	`BurstPredictor` (base: times each prediction, logs each error), `EMAPredictor`, `MLPredictor` (learned model with online refit), `NoisyOraclePredictor` (diagnostic only), `ProcessClassifier`.
scheduler/quantum.py	`compute_quantum` (the percentile rule) and `QuantumController` (epochs, smoothing, error ratios, class modulation, `ratio_percentile` used by the guard).
scheduler/adaptive.py	`priority_score`, `guarded_remaining` (the overrun guard) and `AdaptiveScheduler` (ordering with ageing, optional guard and persistent ageing, uses the controller for the quantum).
scheduler/metrics.py	`per_process` (T, W, R per process), `summarize` (one result row per run), `jain_index`, `count_starved`, `oracle_gap_closed`.
scheduler/configs.py	The 11 configuration names plus Adaptive-ML-G, -A and -GA, default parameters (β = 0.03, p = 80, E = 50, γ = 0.3), the tuned guard (c = 0.25, median ratio), ablation variants, `make_policy(name)` and `run_policy(specs, name)`.
train_models.py	Builds the training data, compares 4 regressors and 2 classifiers, saves the chosen models.
tune_beta.py	Chooses the ageing weight β on separate seeds.
run_experiments.py	Runs the full experiment in parallel across CPU cores and saves raw CSVs.
analyse.py	Turns the CSVs into REPORT.md, a summary table and 8 figures.
demo.py	10-second live demo in three parts (`--part 1/2/3`).
tune_innovation.py	Tunes the guard and persistent ageing on seeds 100–102; also measures how much waiting the ageing reset discards.
run_innovation.py	Prediction-quality sweep (`--part noise`) and the W1–W7 comparison of 14 configurations (`--part compare`); writes `results/innovation/`.
analyse_innovation.py	Paired statistics, `results/innovation/REPORT.md` and the three figures in `figures/innovation/`.
demo_innovation.py	10-second live demo of W7: SJF-EMA, RR-5, MLFQ, Adaptive-ML and Adaptive-ML-G on one instance.
tests/	53 pytest tests: the original 37 plus 16 for the noisy oracle, the guard, persistent ageing, W7 and reproduction of the saved results.

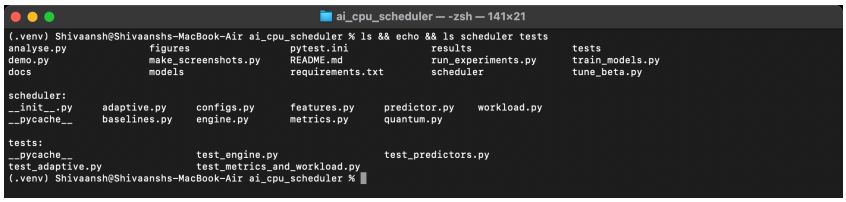


Fig. 3. Project files and the scheduler package

To see how small a policy is, this is the complete Round Robin and SJF code. Round Robin always picks the front of the queue with a fixed quantum; SJF picks the smallest predicted

burst:

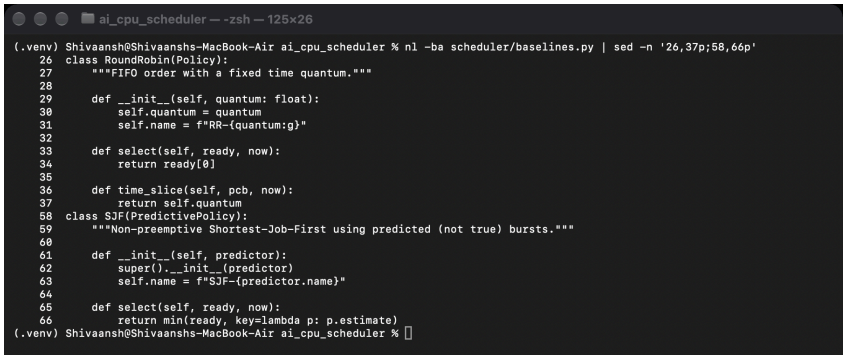


Fig. 4. Round Robin and SJF in scheduler/baselines.py

6. The prediction model

6.1 Features

For every process, 12 features are built only from bursts and I/O waits that have already finished, all of it information the operating system already has: the last 5 burst lengths, the mean and standard deviation of past bursts, the exponential average estimate, the last and mean I/O wait, the number of bursts completed, and the change between the last two bursts. A test checks that changing a process's future bursts does not change its features, which proves there is no leakage.

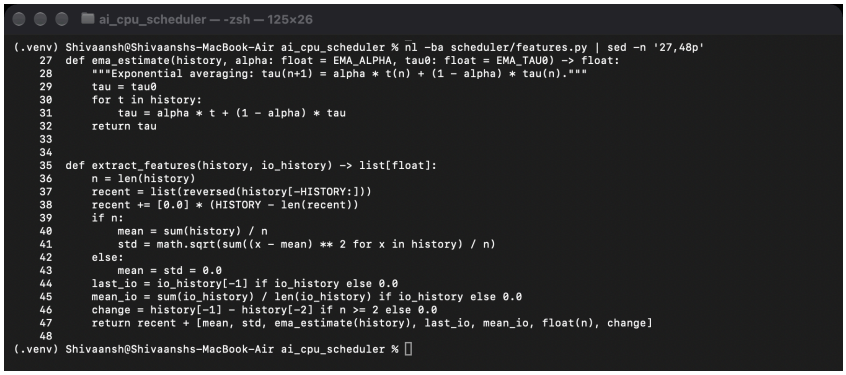


Fig. 5. Exponential averaging and the 12 features in scheduler/features.py

6.2 Training

`train_models.py` generates 2,000 processes (500 of each type) from seeds 1000–1004, which are never used in the experiments. Because the features depend only on a process's own history, the dataset (45,389 rows, one per burst) is built directly without running the simulator. It is split 75/25 **by process**, so no process appears in both training and testing.

Model	MAE (ms)	Latency per prediction	Notes
Exponential averaging	12.36	~0	The baseline to beat
Ridge regression	14.82	0.08 ms	A single linear rule cannot fit all process types
Gradient boosting	12.59	0.50 ms	Large bursts dominate the squared error
Gradient boosting, log target	**10.62**	0.50 ms	**Chosen**: 14% lower error, better for every type

Selection rule: lowest error within a 1 ms latency budget, with Ridge preferred if within 5%. Training on log(burst) helps because burst lengths range from 0.05 ms to hundreds of ms. A logistic regression classifier (macro-F1 0.94) is also saved; a random forest scored 0.94 but was 20 times slower.

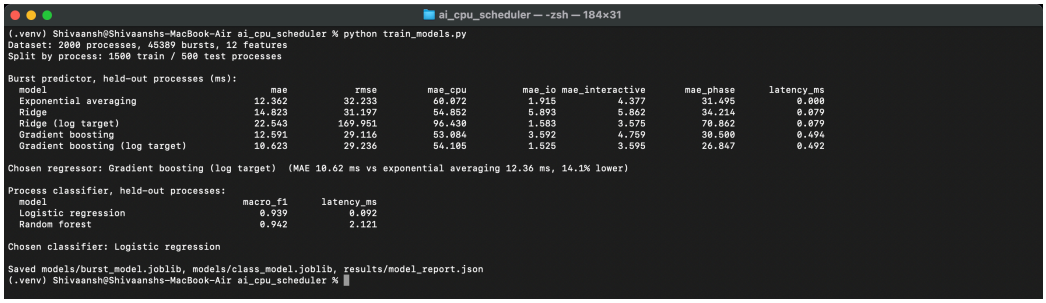


Fig. 6. Output of python train_models.py: model comparison and selection

> **Why single-threaded?** scikit-learn normally uses all CPU cores, but the simulator asks for one prediction at a time. Starting threads for one row made each prediction take 3.7 ms instead of 0.5 ms, so `scheduler/__init__.py` sets `OMP_NUM_THREADS=1`.

6.3 Feedback loop

When a burst finishes, `observe()` compares it with the prediction, logs the error (this is how prediction MAE is reported for every run) and adds the example to a rolling buffer of 4,000. Every 2,000 new examples the model is refitted on 10,000 offline rows plus the buffer. The quantum controller also keeps the last 200 ratios *actual / predicted*.

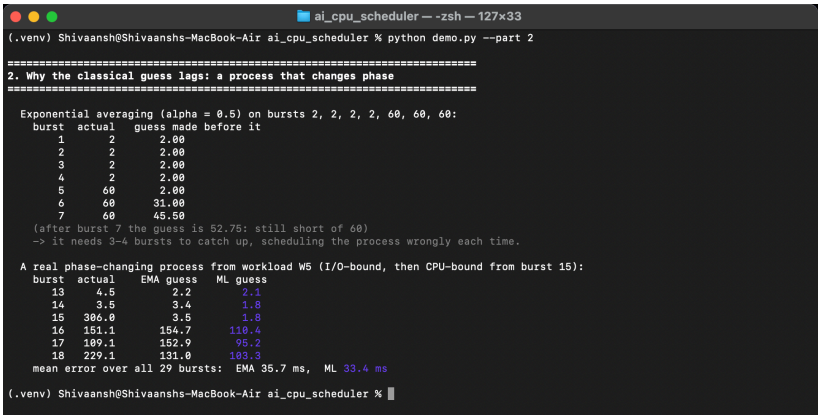


Fig. 7. Output of python demo.py --part 2: exponential averaging lagging, and EMA vs the learned model on a real process

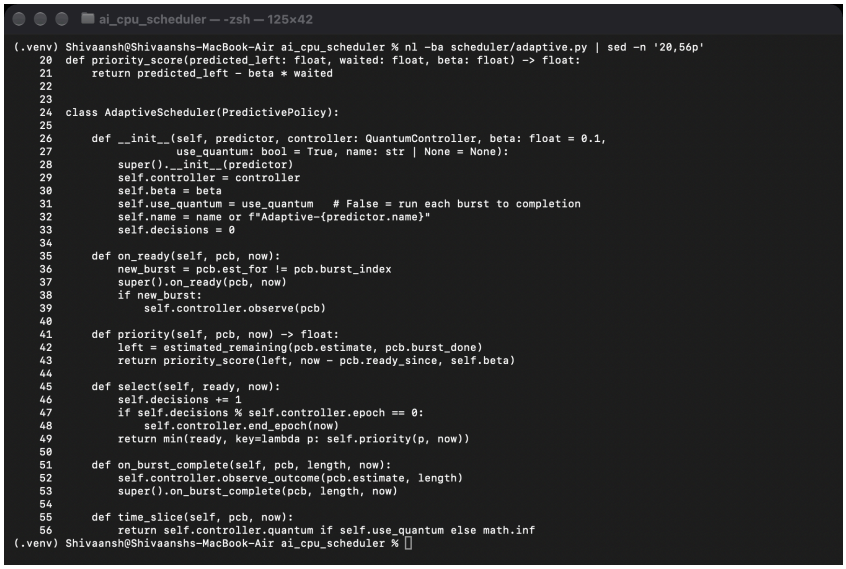
7. The adaptive scheduler in detail

7.1 Who runs next

$$P = \textit{(predicted time left in the burst)} - \beta \times \textit{(time waited in the ready queue)}$$

The lowest P runs. *Predicted time left* is the prediction minus the CPU time already received in this burst; if the burst has already run longer than predicted, the original prediction is used again. **Example:** with β = 0.1, process A predicted at 50 ms that has waited 400 ms has P = 50 − 40 = 10; process B predicted at 5 ms that has waited 10 ms has P = 5 − 1 = 4, so B runs. If A had waited 600 ms, P = −10 and A would run first. With β = 0 the scheduler is exactly SJF (checked by a test). The final value β = 0.03 was picked by `tune_beta.py`.

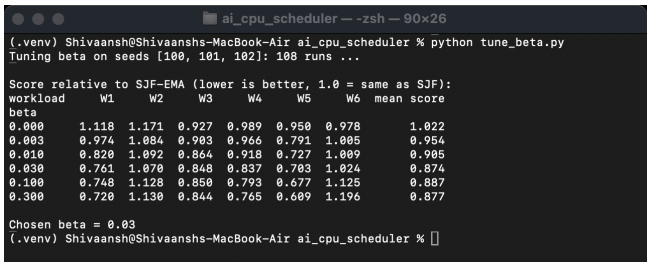
The overrun guard (Adaptive-ML-G). Falling back to the original prediction is exactly what goes wrong when a long burst was predicted as short: after every quantum it returns with the same short estimate. The guard keeps the normal rule until the burst has run longer than prediction × r, where r is the median of the last 200 ratios actual/predicted (at least 1). After that, time left = max(prediction, 0.25 × CPU time used). **Example:** predicted 1.2 ms, already ran 40 ms: time left 10 ms instead of 1.2 ms, so a new interactive burst predicted at 5 ms goes first; after 200 ms the burst looks like 50 ms. It never makes a burst look shorter than its prediction. c = 0.25 was chosen on seeds 100–102.



```
ai_cpu_scheduler — -zsh — 125×42

(.venv) Shivaansh@Shivaanshs-MacBook-Air ai_cpu_scheduler % nl -ba scheduler/adaptive.py | sed -n '20,56p'
    20  def priority_score(predicted_left: float, waited: float, beta: float) -> float:
    21      return predicted_left - beta * waited
    22
    23
    24  class AdaptiveScheduler(PredictivePolicy):
    25
    26      def __init__(self, predictor, controller: QuantumController, beta: float = 0.1,
    27                   use_quantum: bool = True, name: str | None = None):
    28          super().__init__(predictor)
    29          self.controller = controller
    30          self.beta = beta
    31          self.use_quantum = use_quantum   # False = run each burst to completion
    32          self.name = name or f"Adaptive-{predictor.name}"
    33          self.decisions = 0
    34
    35      def on_ready(self, pcb, now):
    36          new_burst = pcb.est_for != pcb.burst_index
    37          super().on_ready(pcb, now)
    38          if new_burst:
    39              self.controller.observe(pcb)
    40
    41      def priority(self, pcb, now) -> float:
    42          left = estimated_remaining(pcb.estimate, pcb.burst_done)
    43          return priority_score(left, now - pcb.ready_since, self.beta)
    44
    45      def select(self, ready, now):
    46          self.decisions += 1
    47          if self.decisions % self.controller.epoch == 0:
    48              self.controller.end_epoch(now)
    49          return min(ready, key=lambda p: self.priority(p, now))
    50
    51      def on_burst_complete(self, pcb, length, now):
    52          self.controller.observe_outcome(pcb.estimate, length)
    53          super().on_burst_complete(pcb, length, now)
    54
    55      def time_slice(self, pcb, now):
    56          return self.controller.quantum if self.use_quantum else math.inf
(.venv) Shivaansh@Shivaanshs-MacBook-Air ai_cpu_scheduler %
```

Fig. 8. The adaptive scheduler in scheduler/adaptive.py: priority with ageing, epoch trigger and quantum



```
ai_cpu_scheduler — -zsh — 90×26

(.venv) Shivaansh@Shivaanshs-MacBook-Air ai_cpu_scheduler % python tune_beta.py
Tuning beta on seeds [100, 101, 102]: 108 runs ...

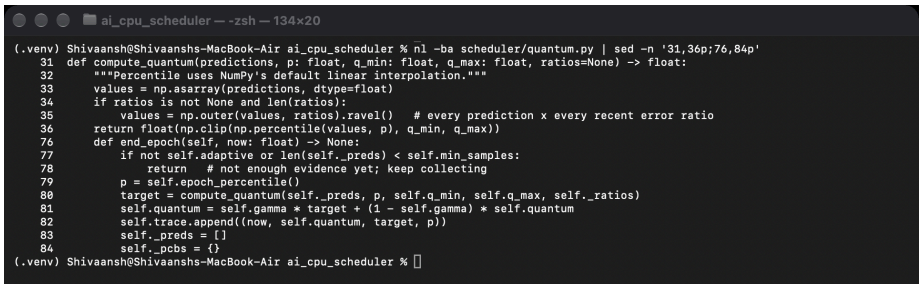
Score relative to SJF-EMA (lower is better, 1.0 = same as SJF):
workload      W1     W2     W3     W4     W5     W6  mean score
beta
0.000      1.118  1.171  0.927  0.989  0.950  0.978       1.022
0.003      0.974  1.084  0.903  0.966  0.791  1.005       0.954
0.010      0.820  1.092  0.864  0.918  0.727  1.009       0.905
0.030      0.761  1.070  0.848  0.837  0.703  1.024       0.874
0.100      0.748  1.128  0.850  0.793  0.677  1.125       0.887
0.300      0.720  1.130  0.844  0.765  0.609  1.196       0.877

Chosen beta = 0.03
(.venv) Shivaansh@Shivaanshs-MacBook-Air ai_cpu_scheduler %
```

Fig. 9. Output of python tune_beta.py: choosing β on separate seeds

7.2 How long it runs

Every 50 decisions, the quantum controller takes the predictions of the bursts that became ready in that period, multiplies each by every recent error ratio, takes the 80th percentile, clamps it to 1–100 ms, and smooths it: $q = 0.3 \times q_{new} + 0.7 \times q$.



```
ai_cpu_scheduler — -zsh — 134×20

(.venv) Shivaansh@Shivaanshs-MacBook-Air ai_cpu_scheduler % nl -ba scheduler/quantum.py | sed -n '31,36p;76,84p'
    31  def compute_quantum(predictions, p: float, q_min: float, q_max: float, ratios=None) -> float:
    32      """Percentile uses NumPy's default linear interpolation."""
    33      values = np.asarray(predictions, dtype=float)
    34      if ratios is not None and len(ratios):
    35          values = np.outer(values, ratios).ravel()   # every prediction x every recent error ratio
    36      return float(np.clip(np.percentile(values, p), q_min, q_max))
    76      def end_epoch(self, now: float) -> None:
    77          if not self.adaptive or len(self._preds) < self.min_samples:
    78              return   # not enough evidence yet; keep collecting
    79          p = self.epoch_percentile()
    80          target = compute_quantum(self._preds, p, self.q_min, self.q_max, self._ratios)
    81          self.quantum = self.gamma * target + (1 - self.gamma) * self.quantum
    82          self.trace.append((now, self.quantum, target, p))
    83          self._preds = []
    84          self._pcbs = {}
(.venv) Shivaansh@Shivaanshs-MacBook-Air ai_cpu_scheduler %
```

Fig. 10. The percentile rule and epoch update in scheduler/quantum.py

Worked example (I/O-bound workload). Bursts average 2 ms, so almost every prediction is about 2 ms. If the quantum used the raw predictions, the 80th percentile would be about 2 ms, too small, because 20% of real exponential bursts with mean 2 ms are longer than 3.2 ms. The recent ratios actual/predicted are spread widely, and for exponential bursts their 80th percentile is about 1.6, so the 80th percentile of 2 × ratios is about **3.2 ms**, matching the textbook rule. Starting from q = 10 ms, smoothing gives 0.3 × 3.2 + 0.7 × 10 = 7.96 ms, then 6.5, 5.5, and so on, settling near 3.5 ms. This error-ratio fix was added after the first version produced quanta that were too small on W2 and W3.

Class modulation. The classifier labels the processes seen in the epoch; the percentile becomes 80 + 10 × (fraction CPU-bound − fraction interactive), limited to 50–95. Interactive mixes get a smaller quantum (faster response), CPU-bound mixes a larger one (fewer switches).

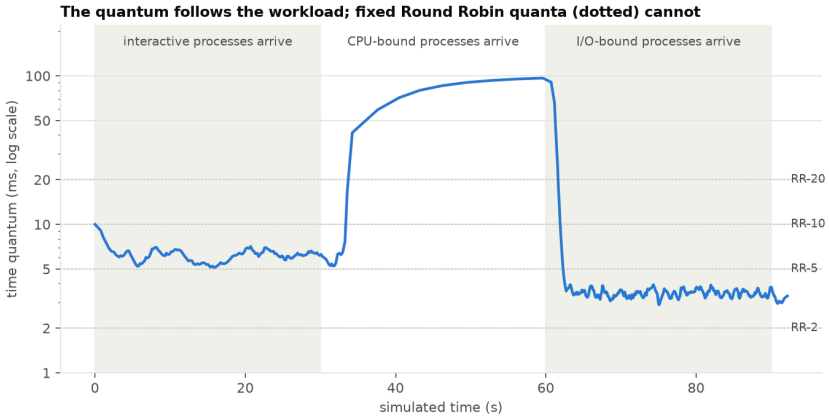


Fig. 11. The quantum on a workload that changes from interactive (about 6 ms) to CPU-bound (close to the 100 ms cap) to I/O-bound (about 3.5 ms)

8. Experiments and evaluation pipeline

Workloads. Four process types: CPU-bound (lognormal bursts, median 100 ms, 3–8 bursts), I/O-bound (exponential, mean 2 ms, 20–60 bursts), interactive (exponential, mean 5 ms, 10–40 bursts) and phase-changing (I/O-bound then CPU-bound). Six profiles of 200 processes at 85% CPU load: W1 CPU, W2 I/O, W3 interactive, W4 mixed (40/30/20/10%), W5 phase-changing, W6 = W4 at 3× arrival rate.

Configurations (11). FCFS, SJF-EMA, SRTF-EMA, RR-2, RR-5, RR-10, RR-20, MLFQ, Oracle, Adaptive-EMA (our scheduler with the classical predictor) and Adaptive-ML (our scheduler).

Runs. 6 workloads × 11 configurations × 10 seeds = **660 paired runs**, plus an ablation (120 runs), sweeps of β, p and E (130 runs) and switch costs of 0.05/0.1/0.5 ms (150 runs). The runs are spread over 9 worker processes and take about 4.5 minutes.

Pipeline.

```
train_models.py  -> models/*.joblib, results/model_report.json
tune_beta.py     -> results/beta_tuning.csv           (chooses beta =
0.03)
run_experiments.py -> results/main_results.csv, ablation.csv, sweeps.csv,
                      switch_cost.csv, quantum_trace.csv,
prediction_trace.csv
analyse.py       -> results/REPORT.md, summary_means.csv,
figures/fig1..fig8
tune_innovation.py -> results/innovation/tuning.csv,
                      ageing_reset_diagnostic.csv  (chooses c = 0.25)
run_innovation.py  -> results/innovation/noise_sweep.csv, comparison.csv
analyse_innovation.py -> results/innovation/REPORT.md, stats.csv,
                         figures/innovation/innov1..3
```

Fairness of the comparison. Every policy sees the same seeds (paired comparison), the oracle gives a reference point, and expected outcomes (H1–H7) were written down before any experiment. Review 1's rule was that an improvement only counts if the mean paired improvement over seeds is bigger than one standard deviation. On top of that, every main claim now has a paired bootstrap 95% confidence interval and an exact Wilcoxon test with Holm correction (Section 9.4).

9. Robustness to prediction errors: the overrun guard, W7 and statistics

9.1 How much prediction quality matters

The noisy oracle gives the scheduler the true burst times random error exp(σZ), for nine values of σ from 0 (perfect) to 2, on W4 and W5 (`python run_innovation.py --part noise`, about 20 s). The same noisy predictions go to the full proposed mechanism and to plain SJF.

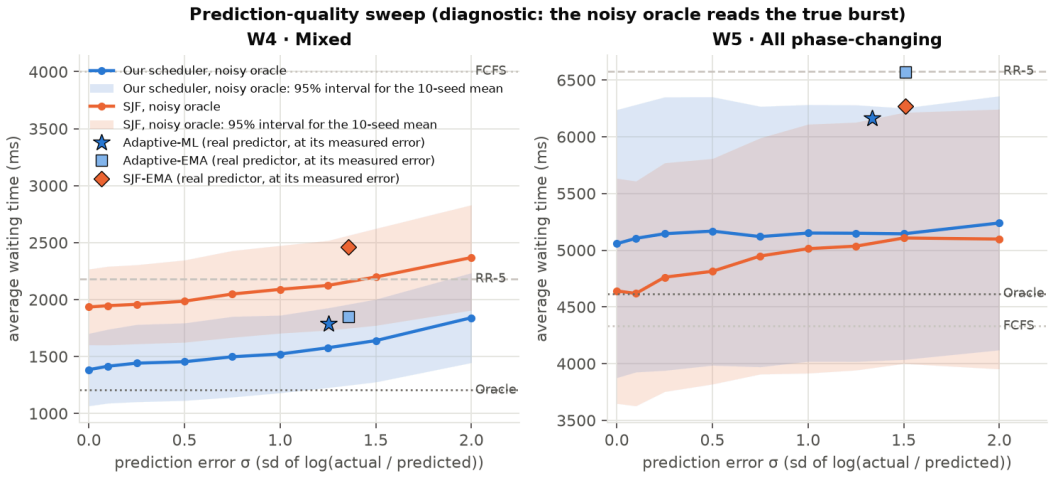


Fig. 12. Average wait as prediction error grows; the real predictors are placed at their measured error

- **W4:** with perfect predictions the mechanism waits 1,384 ms against 1,934 ms for SJF (−28.4%, 10/10 seeds). With the real model it waits 1,786 ms, so perfect predictions would

still save 22.5%.

- **Real errors cost more than random ones.** The model's measured error spread is σ = 1.25, but it performs like σ = 1.86 on the noisy-oracle curve, because real errors are systematic.
- **W5:** the curve is flat (5,057 to 5,240 ms); with perfect predictions the mechanism is 9.0% worse than SJF (4,641 ms), so time slicing is the bottleneck there.

9.2 The adversarial workload W7 and the guard

W7 has 50% interactive, 20% I/O-bound and 30% bait processes. A bait process runs 10–20 bursts of about 1 ms, then one burst with a median of 400 ms, twice, then a few short bursts; the bait processes are 30% of the processes but **76% of the CPU demand** (72–80% across seeds). It was designed to attack the scheduler, not to look like a normal system.

W7, mean of 10 seeds (ms)	Interactive wait	Bait wait	Avg wait (all)	Worst wait	Starved / run
SJF-EMA	4,149	4,169	3,847	13,766	3.4
RR-5	796	3,977	1,741	9,803	0.0
MLFQ	443	4,188	**1,495**	10,536	0.0
Adaptive-ML	1,399	4,378	2,083	21,382	4.0
Adaptive-ML-G	**112**	5,349	1,634	20,283	6.9

The guard lowers the average wait by 21.5% and interactive waiting by 92% (both on 10 of 10 seeds, significant). The bait processes pay: they wait 22% longer, and starved processes rise from 4.0 to 6.9. **MLFQ still has the lowest average wait on W7** (9.3% below the guard), and the worst case with the guard is about twice that of RR-5 and MLFQ.

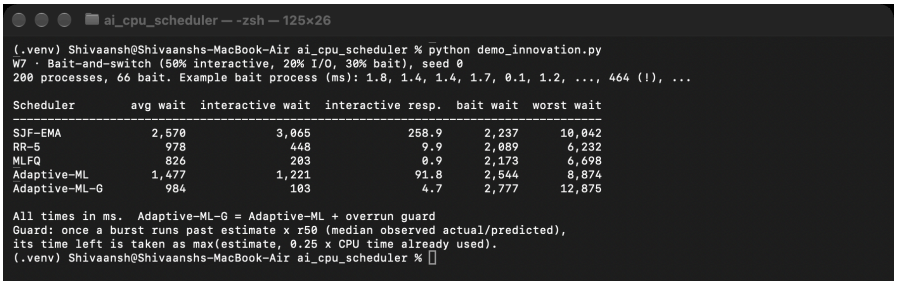


Fig. 13. Real output of python demo_innovation.py (W7, seed 0)

9.3 Cost of the guard elsewhere, and persistent ageing

On W1–W5 the guard changes the average wait by −1.8% to +0.1%, none significant after correction; on W6 it lowers it by 1.6%. The W4 worst case rises by 5.6% (not significant after correction). Separately, the ageing term restarts after every time slice: on the tuning seeds this discards 70% of the waiting on W4 (84% for the ten worst-off processes). A persistent version (Adaptive-ML-A, β = 0.01) improves W2 by 6.9%, but raises the average wait by 17.5% on W6 and 51.2% on W7, because the credit rewards the long bait bursts. **It was not kept.** The recommended configuration is Adaptive-ML-G.

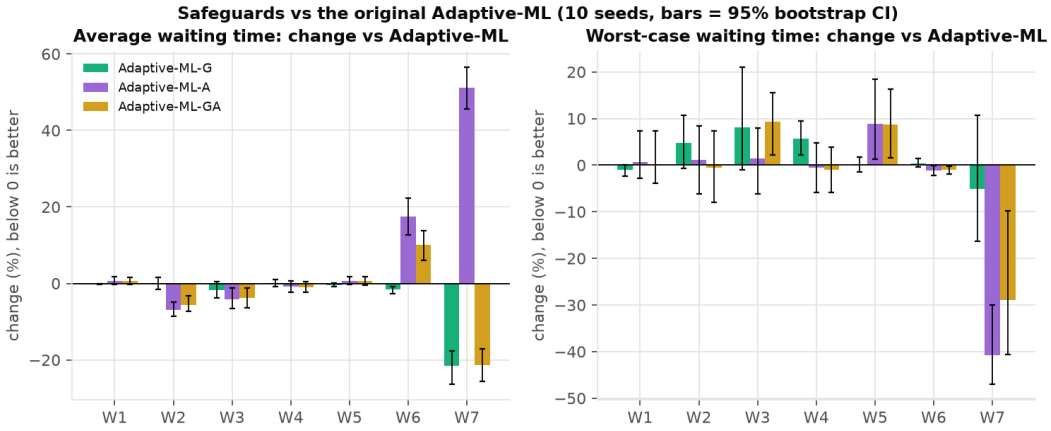
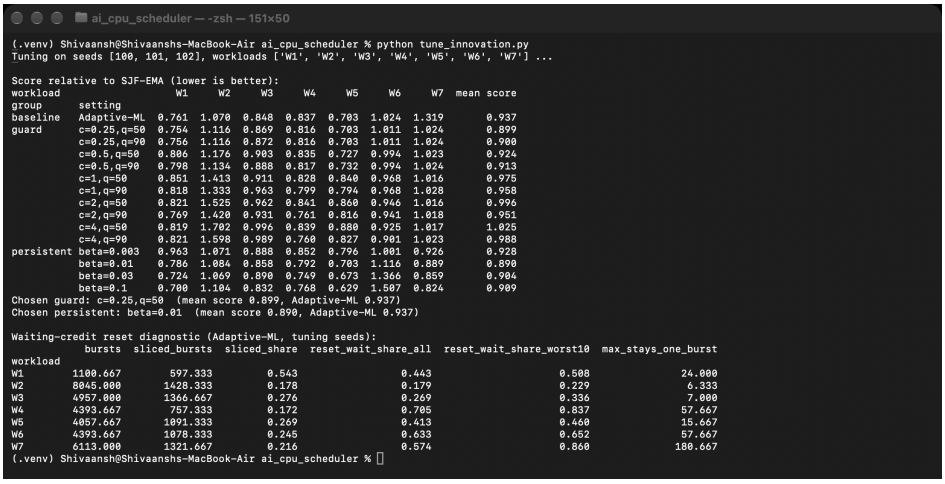


Fig. 14. Each safeguard against Adaptive-ML on W1–W7, with 95% bootstrap confidence intervals



```
ai_cpu_scheduler — -zsh — 151×50

(.venv) Shivaansh@Shivaanshs-MacBook-Air ai_cpu_scheduler % python tune_innovation.py
Tuning on seeds [100, 101, 102], workloads ['W1', 'W2', 'W3', 'W4', 'W5', 'W6', 'W7'] ...

Score relative to SJF-EMA (lower is better):
workload                    W1     W2     W3     W4     W5     W6     W7  mean score
group      setting
baseline   Adaptive-ML   0.761  1.070  0.848  0.837  0.703  1.024  1.319       0.937
guard      c=0.25,q=50   0.754  1.116  0.869  0.816  0.703  1.011  1.024       0.899
           c=0.25,q=90   0.756  1.116  0.872  0.816  0.703  1.011  1.024       0.900
           c=0.5,q=50    0.806  1.176  0.903  0.835  0.727  0.994  1.023       0.924
           c=0.5,q=90    0.798  1.134  0.888  0.817  0.732  0.994  1.024       0.913
           c=1,q=50      0.851  1.413  0.911  0.828  0.840  0.968  1.016       0.975
           c=1,q=90      0.818  1.333  0.963  0.799  0.794  0.968  1.028       0.958
           c=2,q=50      0.821  1.525  0.962  0.841  0.860  0.946  1.016       0.996
           c=2,q=90      0.769  1.420  0.931  0.761  0.816  0.941  1.018       0.951
           c=4,q=50      0.819  1.702  0.996  0.839  0.880  0.925  1.017       1.025
           c=4,q=90      0.821  1.598  0.989  0.760  0.827  0.901  1.023       0.988
persistent beta=0.003    0.963  1.071  0.888  0.852  0.796  1.001  0.926       0.928
           beta=0.01     0.786  1.084  0.858  0.792  0.703  1.116  0.889       0.890
           beta=0.03     0.724  1.069  0.890  0.749  0.673  1.366  0.859       0.904
           beta=0.1      0.700  1.104  0.832  0.768  0.629  1.507  0.824       0.909
Chosen guard: c=0.25,q=50  (mean score 0.899, Adaptive-ML 0.937)
Chosen persistent: beta=0.01  (mean score 0.890, Adaptive-ML 0.937)

Waiting-credit reset diagnostic (Adaptive-ML, tuning seeds):
              bursts  sliced_bursts  sliced_share  reset_wait_share_all  reset_wait_share_worst10  max_stays_one_burst
workload
W1        1100.667        597.333         0.543                 0.443                     0.508               24.000
W2        8045.000       1428.333         0.178                 0.179                     0.229                6.333
W3        4957.000       1366.667         0.276                 0.269                     0.336                7.000
W4        4393.667        757.333         0.172                 0.705                     0.837               57.667
W5        4057.667       1091.333         0.269                 0.413                     0.460               15.667
W6        4393.667       1078.333         0.245                 0.633                     0.652               57.667
W7        6113.000       1321.667         0.216                 0.574                     0.860              180.667
(.venv) Shivaansh@Shivaanshs-MacBook-Air ai_cpu_scheduler %
```

Fig. 15. Real output of python tune_innovation.py: tuning grids on seeds 100–102 and the ageing-reset diagnostic

9.4 Statistical validation

`analyse_innovation.py` compares configurations seed by seed: the change in the mean, a percentile bootstrap 95% confidence interval (10,000 resamples), the exact two-sided Wilcoxon signed-rank test, and a Holm correction within each family of related tests. With ten seeds the smallest possible p-value is 0.002. A result is called significant only if the Holm-corrected p is below 0.05; for example, the guard's +5.6% W4 worst case has unadjusted p = 0.027 but Holm p = 0.328, so it is *not* significant after correction.

```
ai_cpu_scheduler — -zsh — 174×55

(.venv) Shivaansh@Shivaanshs-MacBook-Air ai_cpu_scheduler % grep -A17 '### Safeguard Adaptive-ML-G vs' results/innovation/REPORT.md; grep -A12 '### W7 Adaptive-ML-G vs baseli
nes' results/innovation/REPORT.md
### Safeguard Adaptive-ML-G vs Adaptive-ML

| Workload | Metric | Reference | New | Ref. mean | New mean | Change | 95% CI | p (Wilcoxon) | p (Holm) | Wins |
|---|---|---|---|---|---|---|---|---|---|---|
| W1 | avg_wait | Adaptive-ML | Adaptive-ML-G | 2,725 | 2,720 | -0.2% | [-0.3, -0.0] | 0.027 | 0.328 | 8/10 |
| W2 | avg_wait | Adaptive-ML | Adaptive-ML-G | 450 | 451 | +0.1% | [-1.6, +1.7] | 0.922 | 1.000 | 5/10 |
| W3 | avg_wait | Adaptive-ML | Adaptive-ML-G | 505 | 496 | -1.8% | [-3.7, +0.5] | 0.375 | 1.000 | 5/10 |
| W4 | avg_wait | Adaptive-ML | Adaptive-ML-G | 1,786 | 1,786 | +0.0% | [-0.8, +1.0] | 1.000 | 1.000 | 6/10 |
| W5 | avg_wait | Adaptive-ML | Adaptive-ML-G | 6,163 | 6,139 | -0.4% | [-0.8, +0.0] | 0.160 | 1.000 | 7/10 |
| W6 | avg_wait | Adaptive-ML | Adaptive-ML-G | 17,599 | 17,318 | -1.6% | [-2.7, -0.8] | 0.002 | 0.027 | 10/10 |
| W7 | avg_wait | Adaptive-ML | Adaptive-ML-G | 2,083 | 1,634 | -21.5% | [-26.3, -17.7] | 0.002 | 0.027 | 10/10 |
| W1 | max_wait | Adaptive-ML | Adaptive-ML-G | 12,072 | 11,942 | -1.1% | [-2.4, +0.0] | 0.195 | 1.000 | 5/10 |
| W2 | max_wait | Adaptive-ML | Adaptive-ML-G | 1,311 | 1,374 | +4.8% | [-0.7, +10.7] | 0.322 | 1.000 | 4/10 |
| W3 | max_wait | Adaptive-ML | Adaptive-ML-G | 1,765 | 1,908 | +8.1% | [-1.0, +21.0] | 0.084 | 0.840 | 2/10 |
| W4 | max_wait | Adaptive-ML | Adaptive-ML-G | 18,388 | 19,426 | +5.6% | [+2.2, +9.5] | 0.027 | 0.328 | 2/10 |
| W5 | max_wait | Adaptive-ML | Adaptive-ML-G | 25,831 | 25,876 | +0.2% | [-1.5, +1.7] | 0.492 | 1.000 | 4/10 |
| W6 | max_wait | Adaptive-ML | Adaptive-ML-G | 68,929 | 69,174 | +0.4% | [-0.4, +1.4] | 1.000 | 1.000 | 4/10 |
| W7 | max_wait | Adaptive-ML | Adaptive-ML-G | 21,382 | 20,283 | -5.1% | [-16.3, +10.6] | 0.922 | 1.000 | 4/10 |
### W7 Adaptive-ML-G vs baselines

| Workload | Metric | Reference | New | Ref. mean | New mean | Change | 95% CI | p (Wilcoxon) | p (Holm) | Wins |
|---|---|---|---|---|---|---|---|---|---|---|
| W7 | avg_wait | SJF-EMA | Adaptive-ML-G | 3,847 | 1,634 | -57.5% | [-60.8, -55.1] | 0.002 | 0.018 | 10/10 |
| W7 | wait_interactive | SJF-EMA | Adaptive-ML-G | 4,149 | 112 | -97.3% | [-97.6, -96.9] | 0.002 | 0.018 | 10/10 |
| W7 | max_wait | SJF-EMA | Adaptive-ML-G | 13,766 | 20,283 | +47.3% | [+27.5, +67.1] | 0.004 | 0.018 | 1/10 |
| W7 | avg_wait | RR-5 | Adaptive-ML-G | 1,741 | 1,634 | -6.2% | [-10.1, -1.1] | 0.049 | 0.049 | 8/10 |
| W7 | wait_interactive | RR-5 | Adaptive-ML-G | 796 | 112 | -85.9% | [-88.2, -82.3] | 0.002 | 0.018 | 10/10 |
| W7 | max_wait | RR-5 | Adaptive-ML-G | 9,803 | 20,283 | +106.9% | [+92.5, +118.0] | 0.002 | 0.018 | 0/10 |
| W7 | avg_wait | MLFQ | Adaptive-ML-G | 1,495 | 1,634 | +9.3% | [+4.5, +15.3] | 0.004 | 0.018 | 1/10 |
| W7 | wait_interactive | MLFQ | Adaptive-ML-G | 443 | 112 | -74.7% | [-79.8, -66.2] | 0.002 | 0.018 | 10/10 |
| W7 | max_wait | MLFQ | Adaptive-ML-G | 10,536 | 20,283 | +92.5% | [+77.0, +105.5] | 0.002 | 0.018 | 0/10 |
(.venv) Shivaansh@Shivaanshs-MacBook-Air ai_cpu_scheduler %
```

Fig. 16. Real output: the guard against Adaptive-ML on W1–W7, and against the baselines on W7

10. Results and what they mean

Workload	Avg wait vs SJF-EMA	Worst wait vs SJF-EMA	Seeds better	Mean quantum
W1 CPU-bound	−1.9%	−56%	7/10	92 ms
W2 I/O-bound	+13.8%	+0.5%	0/10	3.5 ms
W3 Interactive	+5.3%	−27%	2/10	6.4 ms
W4 Mixed	**−27.4%**	**−23%**	**10/10**	18 ms
W5 Phase-changing	−1.7%	−53%	4/10	95 ms
W6 Overload	+2.6%	−2%	2/10	14 ms

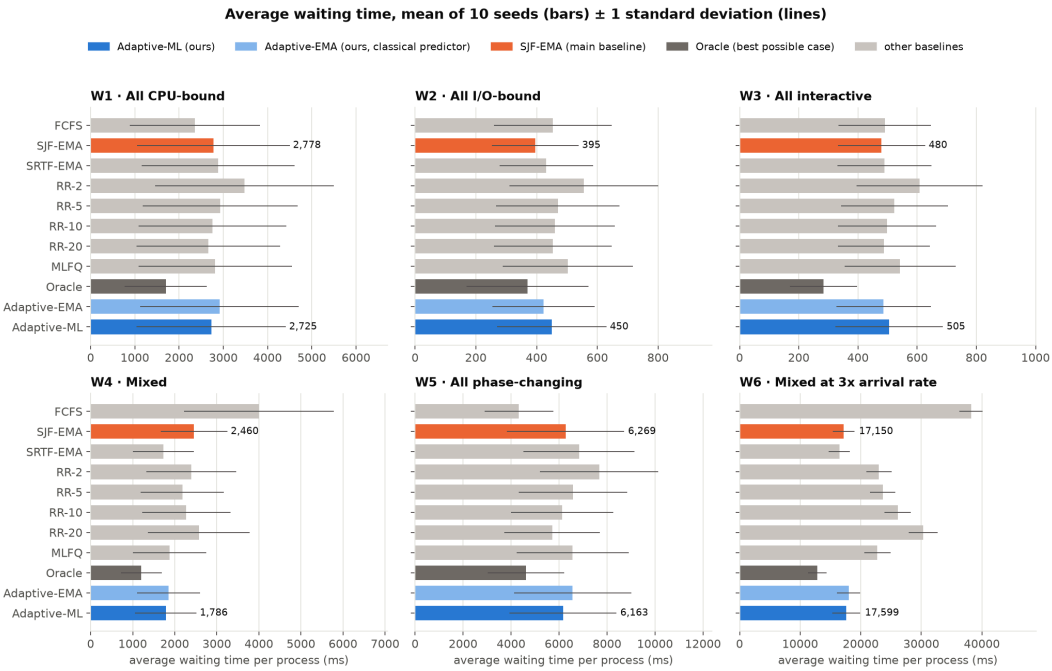


Fig. 17. Average waiting time for all 11 schedulers on the six workloads (10 seeds)

- **Where it helps:** mixed workloads, where short and long bursts compete. On W4 it beats every classical algorithm on average wait except SRTF-EMA (3% better on average, but 35% higher worst case and more switches).
- **Ageing** is what controls the worst case: removing it doubles the worst wait on W5, and starved processes on W5 drop from 7.7 (SJF-EMA) to 0.2.
- **The adaptive quantum** matters most for long bursts: fixing it at 10 ms on W5 costs 11% on average and 52% on worst case.
- **Prediction:** 12–22% lower error than EMA on every workload, but better prediction alone (ML-predicted SJF) gains only 5% on W4. The benefit comes from using the prediction both for ordering and for the quantum.
- **Where it loses:** all-I/O and all-interactive workloads have short random bursts with nothing to predict beyond the mean, so preempting at a small quantum only adds switches. Under overload, the waits are so long that ageing pushes the order towards FCFS.
- **Surprise:** on W5, plain FCFS beats even the oracle on average wait. With many bursts per process, always running the shortest burst first starves processes in their long phase. The oracle is a strong reference, not an absolute lower bound.

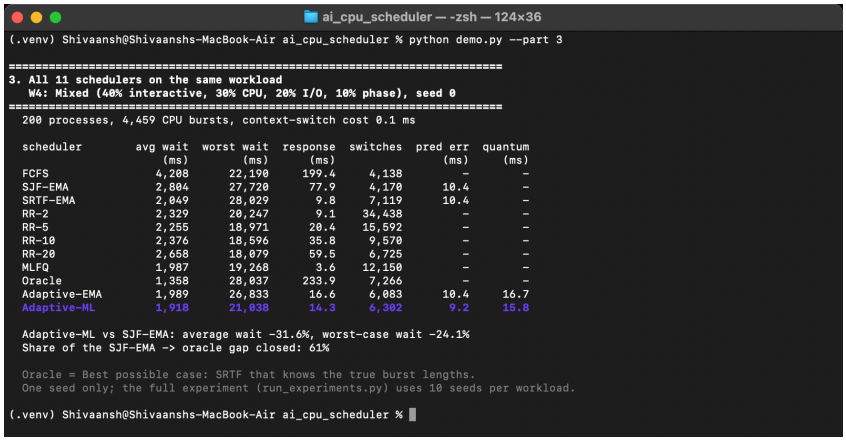


Fig. 18. Output of python demo.py --part 3: all 11 schedulers on the mixed workload (one seed)

11. How to run everything

One-time setup (Python 3.10 or newer; on this laptop it is already done):

```
cd ~/Desktop/ai_cpu_scheduler
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Every time you open a new terminal: `cd ~/Desktop/ai_cpu_scheduler && source .venv/bin/activate`

Command	Time	What it does
`pytest` (or `pytest -v`)	1 s	Runs the 53 tests
`python demo.py`	10 s	Full live demo; add `--part 1`, `2` or `3` for one part, `--workload W5` for another workload
`python train_models.py`	10 s	Retrains and compares the models
`python tune_beta.py`	70 s	Re-tunes β
`python run_experiments.py`	4.5 min	Full experiment; `--quick` for 3 seeds in under a minute
`python analyse.py`	10 s	Rebuilds REPORT.md and the figures
`python demo_innovation.py`	10 s	Live W7 demo: the guard against SJF, RR-5, MLFQ and Adaptive-ML
`python tune_innovation.py`	4.5 min	Re-tunes the guard and persistent ageing on seeds 100–102
`python run_innovation.py`	5 min	Noise sweep and the W1–W7 comparison (`--part noise` or `compare` for one)
`python analyse_innovation.py`	10 s	Rebuilds results/innovation/REPORT.md, stats.csv and figures/innovation/
`sh docs/reviews/build.sh 3`	5 s	Rebuilds Review_3.pdf (run `python3 docs/reviews/src/number_refs.py` first; needs Google Chrome). Without an argument it rebuilds all three reviews.

The demo, part by part. Part 1 shows Gantt charts for four textbook examples and checks the averages (17, 3, 5.67, 6.5 ms). Part 2 shows exponential averaging lagging (2, 31, 45.5, 52.75) and compares EMA with the learned model on a real phase-changing process. Part 3 runs all 11 schedulers on one workload and prints average wait, worst wait, response, switches, prediction error and mean quantum, with Adaptive-ML highlighted.

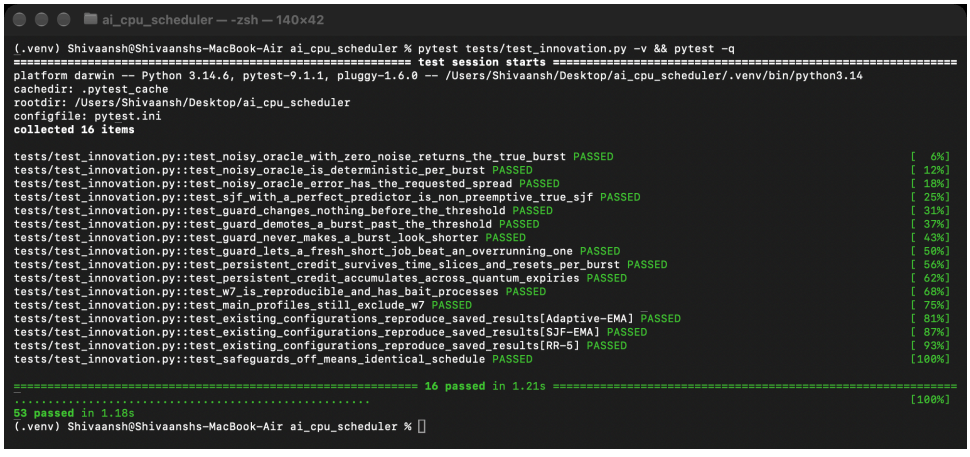


Fig. 19. Real output: the 16 new tests and all 53 tests passing

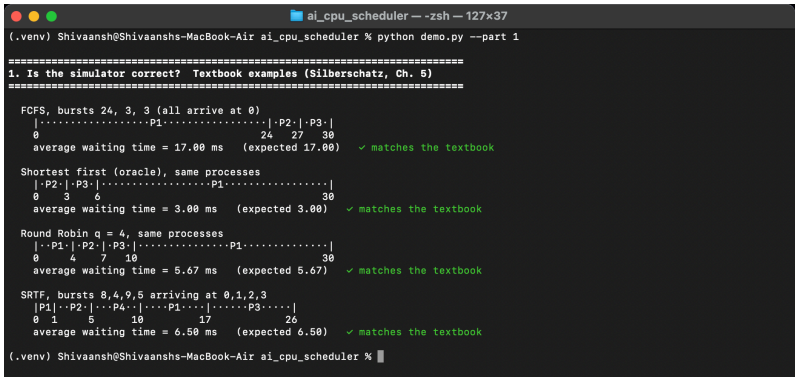


Fig. 20. Output of python demo.py --part 1: textbook checks with Gantt charts

Troubleshooting. "burst_model.joblib not found" means the models are missing: run `python train_models.py`. "command not found: python" or missing modules means the virtual environment is not active: run `source .venv/bin/activate`. Results are deterministic, so re-running gives the same numbers (only timing measurements vary).

12. Where everything is saved

Location	Contents
scheduler/	The simulator, schedulers, predictor and adaptive scheduler
tests/	The 53 tests
models/	burst_model.joblib (predictor), class_model.joblib (classifier)
results/	REPORT.md (every result and verdict), main_results.csv (660 rows, one per run), ablation, sweeps, switch-cost and trace CSVs, model_report.json, run_info.json
results/innovation/	REPORT.md, stats.csv (every paired comparison), comparison.csv (980 runs, W1–W7, 14 configurations), noise_sweep.csv (480 runs), tuning.csv, tuning_round1.csv (superseded guard formula), ageing_reset_diagnostic.csv, run_info.json
figures/innovation/	innov1 prediction-quality sweep, innov2 safeguards against Adaptive-ML, innov3 W7 waiting by process kind
figures/	fig1 average wait, fig2 switches vs response, fig3 worst case, fig4 quantum over time, fig5 prediction example, fig6 ablation, fig7 sweeps, fig8 prediction error
docs/reviews/	Review_1.pdf, Review_2.pdf, Review_3.pdf and their HTML sources
docs/terminal/	Real Terminal screenshots used in the reviews
docs/guide/	This guide and the presentation transcript
docs/ (older)	project_review.pdf, ieee_paper.pdf, rendered screenshots
README.md	Short version of this guide

13. Limitations and future work

- Simulation only: one CPU, no caches or memory effects, no real kernel.
- Synthetic workloads: the model is trained and tested on the same generator (different seeds), which is also why online refit showed no benefit.
- A prediction takes about 0.5 ms in Python, far too slow for a kernel; a small decision tree or lookup table would be needed.
- Linear ageing struggles under heavy overload; scaling it with the load is a natural fix. Making the credit persistent made things worse (Section 9.3).
- The guard is tested only empirically and on one adversarial pattern that we designed (W7, also part of the tuning set on seeds 100–102). MLFQ still beats it on W7's average wait, and its W7 worst case is about twice that of RR-5 and MLFQ.
- Next steps: real burst traces, moving the quantum controller into xv6, and multi-core scheduling.

14. Questions to be ready for

Why use machine learning at all?

Exponential averaging uses one feature and lags after phase changes. The model uses 12 features and is 12–22% more accurate. But the honest finding is that most of the gain comes from using the prediction twice, not from accuracy alone (Adaptive-EMA vs Adaptive-ML shows this).

How do you know the simulator is correct?

It reproduces textbook results exactly (17, 3, 5.67, 6.5 ms), every scheduler delivers the same total CPU time, event times never go backwards, and all 53 tests pass. Re-running the 11 original configurations also reproduces all 660 saved results exactly.

What if the prediction is wrong?

A wrong prediction only affects ordering and quantum; the process still runs. Ageing makes a long-waiting process go first eventually, the quantum uses observed error ratios, and the overrun guard stops a burst that has clearly outrun its prediction from being treated as short. Section 9 measures this directly.

Why not just use SRTF?

SRTF-EMA is 3% better on average on W4 but has a 35% higher worst-case wait and more switches, because it preempts whenever a newly ready process has a shorter predicted remaining time, and it has no ageing.

How did you avoid tuning to the test data?

The model was trained on seeds 1000–1004; β, the guard and persistent ageing were tuned on seeds 100–102; the evaluation used seeds 0–9. All the other parameters came from the original plan.

Why did you add the overrun guard?

Because a burst that outruns its prediction keeps its short estimate and is picked again after every quantum. On W7 this made the scheduler lose to Round Robin and MLFQ. The guard cuts W7's average wait by 21.5% and interactive waiting by 92%, with no significant change on W1–W5.

Isn't W7 designed to make the guard look good?

W7 is designed to expose the weakness, and the documents say it is adversarial. The guard still does not win everywhere on it: MLFQ has a 9.3% lower average wait and a much lower worst case. Its parameter was tuned on seeds 100–102 and the results come from seeds 0–9.

Why not keep persistent ageing?

It matches the stated intent better, but it raised average waits by 17.5% on W6 and 51.2% on W7, because it rewards long mispredicted bursts.

Why does it lose on I/O-bound workloads?

Bursts are short and random (exponential), so nothing can be predicted beyond the mean, and preempting at a small quantum adds context switches without improving the order.